

The bottleneck k -MST

Abraham P. Punnen^{a,*,1,2}, Olena Chapovska^b

^a Department of Mathematics, Simon Fraser University, 13450 102nd AV, Surrey, British Columbia, V3T 5X3, Canada

^b Department of Mathematical Sciences, University of New Brunswick, Saint John, NB E2L 4L5, Canada

Received 7 June 2004

Available online 1 July 2005

Communicated by F. Dehne

Abstract

The k -MST is a well known NP-hard problem and several approximation algorithms exist to solve this problem with a guaranteed performance bound. A closely related problem, called the bottleneck k -MST (BMST(k)) can however be solved in $O(m \log n)$ time on graph with n nodes and m edges. We propose two algorithms to solve BMST(k), one of complexity $O(m + n \log n)$ and the other of $O(m)$ time. We also consider a generalization of BMST(k) which subsumes many bottleneck problems studied in the literature and show that this generalized problem can also be solved in $O(m)$ time.

© 2005 Elsevier B.V. All rights reserved.

Keywords: Graph algorithms; Analysis of algorithms; Computational complexity; Bottleneck problems; k -MST

1. Introduction

Let G be a graph with node set $V(G)$ and edge set $E(G)$. For each edge $e \in E(G)$, a non-negative cost c_e is prescribed. Let $\mathbb{F}$ be the family of all trees of G containing exactly k nodes. Then, the k -MST problem (MST(k)) is to

$$\text{Minimize } \sum_{e \in E(T)} c_e,$$

subject to $T \in \mathbb{F}$.

If $k = |V(G)|$, MST(k) reduces to the well-known minimum spanning tree problem which is solvable in polynomial time. However, for arbitrary k , the MST(k) is NP-hard. The problem MST(k) has been studied extensively in literature primarily from the point of view of approximation algorithms. The problem remains NP-hard even if $c_e \in \{1, 2, 3\}$ for all $e \in E(G)$ and $G = K_n$. However, MST(k) is solvable in polynomial time if the edge costs take two distinct values only and $G = K_n$. For an excellent survey of cardinal-

* Corresponding author.

E-mail addresses: punnen@unbsj.ca (A.P. Punnen), olena@genesys.ca (O. Chapovska).

¹ This work was partially supported by the NSERC discovery grant 0170381 awarded to Abraham P. Punnen.

² This work was completed while the author was at University of New Brunswick, Saint John.

ity constrained combinatorial optimization problems, we refer to [5]. Some typical applications of $MST(k)$ can be found in [3,7].

The survey paper [5] also considers the bottleneck version of $MST(k)$ denoted $(BMST(k))$ which is defined as

Minimize $\max_{e \in T} \{c_e\}$,
subject to $T \in \mathbb{F}$.

For $k = |V(G)|$, $BMST(k)$ reduces to the well known bottleneck spanning tree problem [2]. For fixed k , one can consider all induced subgraphs of G of size k and solve a bottleneck problem on each of these graphs. The best bottleneck solution so generated is an optimal solution to $BMST(k)$. For arbitrary k this brute force method is exponential and not practical. As noted in [5], $BMST(k)$ can be solved using the general threshold approach for bottleneck problems. If $m = |E(G)|$ and $n = |V(G)|$, the binary search version of the threshold algorithm yields an $O(m \log n)$ algorithm for $BMST(k)$ and this is the best known time bound for the problem.

It is well known that every minimum spanning tree of G is also an optimal solution to the bottleneck spanning tree problem on G . Generalizing this result, we show that for any minimum spanning tree of G , there exists a subtree which is an optimal solution to $BMST(k)$. This leads to an $O(m + n \log n)$ algorithm to solve $BMST(k)$. Camerini [2] showed that the binary search version of the threshold algorithm combined with graph contraction can be used to obtain an $O(m)$ algorithm to solve the bottleneck spanning tree problem. This approach do not extend directly to solve $BMST(k)$. However, by considering a generalization of $BMST(k)$, we show that Camerini's approach can be generalized to solve this new problem in $O(m)$ time and hence we have an $O(m)$ algorithm for $BMST(k)$ as well.

Without loss of generality we assume hereafter that all edge costs are distinct. Otherwise we could number the edges arbitrarily and consider the edge costs as an ordered pair with first element as the given cost and the second element as the edge number and lexicographic order to compare these pairs.

2. The $BMST(k)$

We first prove some general results. Let S be a subset of $V(G)$ and T be a spanning tree of G . A subtree T' of T is said to *cover* S if $S \subseteq V(T')$. A subtree T'' of T that covers S is *minimal*, if there is no proper subtree T''' of T'' that covers S .

Let T^0 be a tree in G containing at least one edge and let T^* be a minimum spanning tree of G . Let $\hat{T}$ be a minimal subtree of T^* that covers $V(T^0)$. Note that $V(T^0) \subseteq V(\hat{T})$ and $V(\hat{T}) \setminus V(T^0)$ may or may not be empty.

Theorem 1. $\max_{e \in T^0} \{c_e\} \geq \max_{e \in \hat{T}} \{c_e\}$.

Proof. Choose an edge $e_d \in \hat{T}$ such that $c_{e_d} = \max_{e \in \hat{T}} \{c_e\}$. If $e_d \in T^0$, the result follows immediately. Suppose $e_d \notin T^0$. Since $\hat{T}$ is a minimal subtree of T^* covering $V(T^0)$, there exists two nodes u and v in $V(T^0)$ such that e_d lies on the unique path $P[u, v]$ in $\hat{T}$ joining nodes u and v and $P[u, v]$ contains no nodes of $V(T^0)$ other than u and v . Let $\Pi[u, v]$ be the unique path in T^0 joining u and v . It can be verified that $\hat{T}$ is a minimum spanning tree of the subgraph $\hat{G}$ of G induced by $V(\hat{T})$. Introducing the path $\Pi[u, v]$ to $\hat{T}$ creates one or more cycles in $\hat{T}$ and one of the fundamental cycles C^d so generated contains the edge e_d (see Fig. 1). This fundamental cycle contains ex-

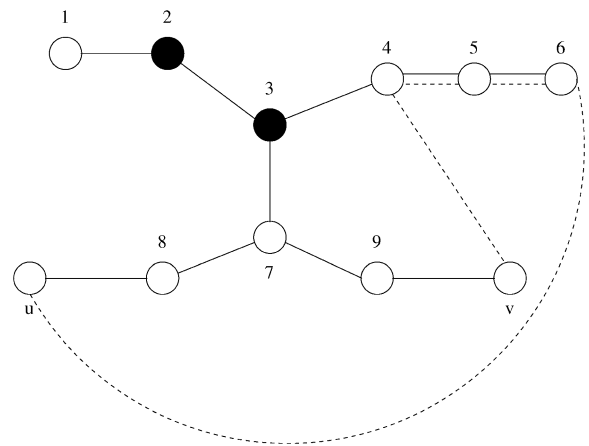


Fig. 1. Example tree $\hat{T}$ in proof of Theorem 1. Dotted lines are members of $\Pi[u, v]$ from T^0 . The dark nodes are those not from T^0 . The fundamental cycle C^d containing $e_d = (7, 8)$ is $u-6-5-4-3-7-8-u$. If $e_d = (7, 9)$ the cycle is $7-9-v-4-3-7$.

actly one edge of T^0 , say e_γ that is not in $\hat{T}$. Since $\hat{T}$ is a minimum spanning tree of $\hat{G}$,

$$c_{e_d} \leq c_{e_\gamma}. \quad (1)$$

But $c_{e_\gamma} \leq \max_{e \in T^0} \{c_e\}$. The result now follows in view of (1). $\square$

Theorem 1 yields the following result.

Theorem 2. *There exists a subtree of T^* which is an optimal solution to BMST(k).*

Proof. Let T^0 be an optimal solution to BMST(k) and T^{**} be the minimal subtree of T^* that covers $V(T^0)$ in T^* . From Theorem 1,

$$\max_{e \in T^{**}} \{c_e\} \leq \max_{e \in T^0} \{c_e\}. \quad (2)$$

Note that $|V(T^{**})| \geq |V(T^0)| = k$. If T^{**} contains more than k -nodes, we can trim the tree T^{**} by removing pendant nodes in sequence to obtain a subtree $\hat{T}$ of T^{**} such that $|V(\hat{T})| = k$. Now from (2) $\max_{e \in \hat{T}} \{c_e\} \leq \max_{e \in T^0} \{c_e\}$. Thus $\hat{T}$ must also be an optimal solution to BMST(k). $\square$

Since a minimum spanning tree can be computed in $O(m + n \log n)$ time [1], an optimal solution to BMST(k) can be obtained in $O(m + n \log n)$ time by searching over the minimum spanning tree for an optimal subtree. It may be noted that there are faster algorithms to compute a minimum spanning tree. A straightforward search scheme over a minimum spanning tree T^* takes $O(n \log n)$ time identify a subtree which is an optimal solution to BMST(k). If such a search scheme is used, then we cannot improve the running time beyond $O(m + n \log n)$ time, no matter what minimum spanning tree algorithm we use. It is possible develop more sophisticated search scheme that runs in $O(n)$ time. One such example is to apply the algorithm in the next section on T^* .

3. A linear time algorithm

We now show that without computing a minimum spanning tree, we can solve BMST(k) in $O(m)$ time for any k . This algorithm can be viewed as an interesting generalization of the algorithm for the bottleneck

spanning tree problem discussed in [2]. Although for dense graphs this worst case bound is the same as the simple bound given in the previous section, for sparse graphs this achieves a better time bound.

For any real number α , let $G(\alpha)$ denotes the spanning subgraph of G , where $E(G(\alpha)) = \{e \in E(G) \mid c_e \leq \alpha\}$. Let $V_1, V_2, \dots, V_p$ be a partition of $V(G)$ (i.e., $V_i \cap V_j = \emptyset$ for $i \neq j$ and $\bigcup_{i=1}^p V_i = V(G)$). Then the collapsed graph $\bar{G}$ of G with respect to the partition $V_1, V_2, \dots, V_p$ of $V(G)$ is the graph with node set $V(\bar{G}) = \{1, 2, \dots, p\}$ and the edge set $E(\bar{G})$ such that $(i, j) \in E(\bar{G})$ if and only if $(u, v) \in E(G)$ for some $u \in V_i$ and $v \in V_j$. The cost $\bar{c}_{ij}$ of the edges (i, j) in $E(\bar{G})$ is defined as

$$\bar{c}_{ij} = \min\{c_{uv} \mid u \in V_i, v \in V_j\}. \quad (3)$$

Let $G_{\alpha_1}, G_{\alpha_2}, \dots, G_{\alpha_p}$ be the connected components of $G(\alpha)$. If $|V(G_{\alpha_i})| < k$ for all i , then the optimal objective function value of BMST(k) is greater than α . In this case if we can redefine BMST(k) on the collapsed graph $\bar{G}(\alpha)$ and then by a suitable choice of α , the problem size could be reduced by half provided the cardinality constraint can be transferred to $\bar{G}(\alpha)$ efficiently. To achieve this easily, we in fact consider the following generalization of BMST(k).

For $i \in V(G)$, let $w_i \geq 0$ be a prescribed weight. Consider the problem:

$$\begin{aligned} BT(k): \text{ Minimize } \max_{e \in T} \{c_e\}, \\ \text{subject to } T \in \mathbb{F}', \end{aligned}$$

where $\mathbb{F}' = \{T \mid T \text{ is a tree in } G \text{ and } \sum_{i \in V(T)} w_i \geq k\}$.

For simplicity, we sometimes use the notation $i \in T$ instead of $i \in V(T)$ when the context is clear. If $w_i = 1$ for all i , then BT(k) reduces to BMST(k) since if the solution to BT(k) contains more than k nodes, we can trim the solution tree to obtain an optimal solution to BMST(k). Before discussing our solution procedure for BT(k) it may be noted that BT(k) generalizes several well known bottleneck problems. For example, let s and t be two fixed nodes in G . Then a bottleneck s – t path problem [8,6] can be solved as a BT(2) by choosing $w_s = w_t = 1$ and $w_i = 0$ for all $i \neq s, t$. The solution of resulting BT(2) may or may not be an s – t path but it will contain the nodes s and t . If it is not an s – t path, we could remove some branches from the solution tree repeatedly to obtain the optimal s – t path. Let S be a given subset of $V(G)$. Choose $w_i = 1$ for

all $i \in S$ and $w_i = 0$ for all $i \in V(G) \setminus S$. Also let $k = |S|$. Then an optimal solution to $BT(k)$ is also an optimal solution to the bottleneck Steiner tree problem [4] on G .

Let $BOT(G, c, w, k)$ denotes the optimal objective function value of $BT(k)$ on the graph G with edge cost function c and node weight function w . The following theorem is key to our linear time algorithm to solve $BT(k)$.

Theorem 3. For any real number α , let $G_{\alpha_1}, G_{\alpha_2}, \dots, G_{\alpha_p}$ be the connected components of $G(\alpha)$.

(i) If $\sum_{j \in G_{\alpha_i}} w_j < k$ for all i , then

$$BOT(G, c, w, k) = BOT(\bar{G}(\alpha), \bar{c}, \bar{w}, k),$$

where $\bar{c}_{ij}$ is defined as in (3) by choosing $V_i = V(G_{\alpha_i})$ and $\bar{w}_i = \sum_{j \in G_{\alpha_i}} w_j$ for $i = 1, \dots, p$.

(ii) If $\sum_{j \in G_{\alpha_i}} w_j \geq k$ for at least one i , then

$$BOT(G, c, w, k)$$

$$= \min\{BOT(G_{\alpha_i}, c^i, w^i, k) \mid i \in B\},$$

where $B = \{i \mid \sum_{j \in G_{\alpha_i}} w_j \geq k\}$ and c^i and w^i are the edge cost and node weight functions for G_{α_i} which are precisely restrictions of c and w to G_{α_i} .

Proof. If $\sum_{j \in G_{\alpha_i}} w_j < k$ for all i , no G_{α_i} contains a feasible solution. Thus an optimal solution to $BT(k)$ must use an edge of cost $> \alpha$. By the definition of $\bar{w}_i, \bar{c}_{ij}$, and the construction of $\bar{G}(\alpha)$, it can be verified that for any tree $T_1 \in G$ with $\sum_{e \in T_1} w_i \geq k$ and $\max_{e \in T_1} c_e > \alpha$ we can find a tree, say $\bar{T}_1$ in $\bar{G}(\alpha)$ such that $\max_{e \in \bar{T}_1} \{\bar{c}_e\} = \max_{e \in T_1} \{c_e\}$ and $\sum_{i \in \bar{T}_1} \bar{w}_i \geq k$. Conversely for any tree $\bar{T}_2$ in $\bar{G}(\alpha)$ with $\sum_{i \in \bar{T}_2} \bar{w}_i \geq k$ we can find at least one tree T_2 in G such that $\sum_{i \in T_2} w_i \geq k$ and $\max_{e \in \bar{T}_2} \bar{c}_e = \max_{e \in T_2} c_e > \alpha$. Thus proof of (i) follows.

To prove (ii), if $\sum_{j \in G_{\alpha_i}} w_j \geq k$ for at least one i , the optimal objective function value is less than or equal to α . Any connected component G_{α_i} with $\sum_{j \in G_{\alpha_i}} w_j < k$ will not lead to a feasible solution. Further a node in such a component can be part of a feasible solution only if an edge of cost more than α

is used. Such a solution cannot be optimal if $|B| \neq \emptyset$. This proves (ii). $\square$

Based on Theorem 3, we now present our $O(m)$ algorithm to solve $BT(k)$. The algorithm uses Theorem 3 repeatedly by choosing α as the median of the edge costs of the graph under consideration. We assume that a function $LB(G)$ is available which with input a graph G along with its node weights and edge costs, computes a lower bound for the optimal objective function value of $BT(k)$ on G . Good lower bounds may reduce the average performance of the algorithm. For the purpose of worst case analysis we could set $LB(G) = -\infty$ for any G making the lower bound function redundant. A formal description is given in Algorithm 1.

Theorem 4. Algorithm LINEAR solves $BT(k)$ in $O(m)$ time on a graph with m nodes.

Proof. The algorithm works in two major parts. The first part is iterative in nature and computes the optimal objective function value. Using this value, an optimal solution is constructed in part 2. The algorithm first computes the median α of the edge costs and constructs the graph $G(\alpha)$. If condition (i) of Theorem 3 is satisfied then computing the optimal objective function value on G is equivalent to computing the same for the modified $BT(k)$ on the collapsed graph $\bar{G}(\alpha)$ as indicated in Theorem 3. If condition (i) of Theorem 3 is not satisfied, then condition (ii) is satisfied. Let $G_1(\alpha), G_2(\alpha), \dots, G_p(\alpha)$ be the connected components of $G(\alpha)$. In this case the optimal solution will be a subtree of one of the graphs $G_i(\alpha)$ where $i \in B = \{j: \sum_{r \in G_j(\alpha)} w_r \geq k\}$. Thus the optimal objective function value can be computed by applying this idea repeatedly on the candidate subgraphs generated as done in the algorithm. Once the optimal objective function value is identified, say α^* , an optimal solution can be identified as any spanning tree of a component of $G(\alpha^*)$ that satisfy the node weight constraint. The construction guarantees that such a component exists.

Let us now look at the complexity of algorithm. It is well known that the median of m numbers can be identified in $O(m)$ time. If α is the median of the edge costs, then $|E(\bar{G}(\alpha))| \leq |E(G)|/2$ and $\sum_{i \in B} |E(G_{\alpha_i})| \leq |E(G)|/2$. Thus in either case, the problem size is reduced by half. Using this analysis

Algorithm LINEAR**begin**BestValue = ∞ stack = $\{G\}$ **while** stack $\neq \emptyset$ **do**choose a graph G from stack.stack = stack – $\{G\}$ S = set of costs of the edges in G **If** $|S| = 1$ **then**Let e_0 be the unique element in S .If BestValue $> c_{e_0}$ then BestValue = c_{e_0} **else** α = median of S {* If $|S|$ is even there are two medians and we choose α as the average of these two values. *} }Construct the graph $G(\alpha)$ Let $G_1(\alpha), G_2(\alpha), \dots, G_p(\alpha)$ be the connected components of $G(\alpha)$ **If** $\sum_{j \in G_i(\alpha)} w_j < k$ for all $i = 1, \dots, p$ **then**construct $\tilde{G}(\alpha)$ **If** $\text{LB}(\tilde{G}(\alpha)) < \text{BestValue}$ **then** stack = stack $\cup \{\tilde{G}(\alpha)\}$ **else****for** $i = 1 \dots p$ **do****if** $\text{LB}(G_i(\alpha)) < \text{BestValue}$ **then** stack = stack $\cup \{G_i(\alpha)\}$ **endfor****endif****endif****endwhile** α = BestValueConstruct $G(\alpha)$ Choose a component $\hat{G}(\alpha)$ of $G(\alpha)$ with $\sum_{i \in V(\hat{G}(\alpha))} w_i \geq k$.

{* by construction at least one such component exists *} }

output any spanning tree of $\hat{G}(\alpha)$.**end**

Algorithm 1.

repeatedly, it can be shown that an optimal objective function value can be obtained in $O(m + m/2 + m/2^2 + \dots) = O(m)$ time. In this time analysis we assumed that the complexity of the lower bounding oracle $\text{LB}(G)$ is constant or equivalently, the lower bound computations are ignored. The final phase of the algorithm of computing the optimal solution given the optimal objective function value is clearly $O(m)$. This completes the proof. $\square$

By choosing $w_i = 1$ for all $i \in V(G)$ algorithm LINEAR solves $\text{BMST}(k)$ in $O(m)$ time. Note that for this instance of $\text{BMST}(k)$ the solution produced by algorithm LINEAR may contain more than k edges. In

this cases we can trim the solution tree to obtain a tree on k edges which is an optimal solution to $\text{BMST}(k)$. It is interesting to note that in $\text{BT}(k)$ if we replace the cardinality constraint be an equality, the resulting problem is NP-hard.

There are several ways to improve the average performance of algorithm LINEAR. Some of these modifications may however increase the worst case complexity. The median can be replaced by $(c_{\max} + c_{\min})/2$ where $c_{\max}$ is the largest edge cost in the graph under consideration and $c_{\min}$ is the smallest edge cost. This choice could reduce some computations in the median calculations but the worst case bound will no longer linear. The algorithm LINEAR can be viewed

as a ‘nice’ tree search (branch and bound) procedure by managing the ‘stack’ as a tree. Clearly this tree will not generate more than n nodes. Effective lower bounding schemes could help pruning lots of branches from the search tree. The order in which the tree search is carried out could also affect the average performance. These are all important considerations in an experimental analysis of the algorithm which is not addressed in this paper.

References

- [1] R.K. Ahuja, T.L. Magnanti, J.B. Orlin, *Network Flows: Theory, Algorithms, and Applications*, Prentice Hall, New York, 1993.
- [2] P.M. Camerini, The min–max spanning tree problem and some extensions, *Inform. Process. Lett.* 7 (1978) 10–14.
- [3] S.Y. Cheng, A. Kumar, Efficient quorumcast routing algorithms, in: *Proceedings of INFOCOM 94*, IEEE, 1994, pp. 840–847.
- [4] C.W. Duin, A. Volgenant, The partial sum criterion for the Steiner trees in graphs and shortest paths, *Europ. J. Operat. Res.* 97 (1997) 172–182.
- [5] M. Ehrgott, H.W. Hamacher, M. Maffioli, Fixed cardinality combinatorial optimization problems, Technical Report, University of Kaiserslautern, 1999.
- [6] H.N. Gabow, R.E. Tarjan, Algorithms for two bottleneck optimization problems, *J. Algorithms* 9 (1988) 411–417.
- [7] H.W. Hamacher, K. Joernsten, Optimal relinquishment according to the Norwegian petrol law: A combinatorial optimization approach, Technical Report no. 7/93, Norwegian School of Economics and Business Administration, Bergen, 1993.
- [8] A.P. Punnen, A linear time algorithm for the maximum capacity path problem, *Europ. J. Oper. Res.* 53 (1991) 402–404.
- [9] C. Blum, M. Ehrgott, Local search algorithms for k -cardinality tree problem, *Discrete Appl. Math.* 128 (2003) 511–540.

Further reading